

RESTAURANT ORDER GUI

CSCI-C 322 Object-Oriented Software Methods

Group 16 Final Project Presentation

Aden Yu & Justin Mabry



PROJECT DESCRIPTION

Restaurant Order GUI is a Java-based ordering system that simulates a standard restaurant workflow:

1. Customers select menu items
2. Orders are created and tracked
3. Payments are processed using different payment methods
4. Order status updates from placed to completed

Workflow: Select Menu Items → Create Order → Process Payment → Track Status → Complete Order

Purpose: Demonstrate object-oriented design patterns in a realistic restaurant ordering system.

DEVELOPMENT TOOLS & LANGUAGES

Java: Main programming language

JavaFX: GUI framework for the restaurant order window

Maven: Build tool used to manage JavaFX dependencies

IntelliJ IDEA: IDE used to run and test the project

GitHub: Repository and version control

MAIN FEATURES

Implemented features

- Menu list shows Pizza, Burger, and Drink items with prices
- Add Item button creates menu objects through the factory
- Order area displays selected items and order total
- Cash and card buttons select the payment strategy
- Status buttons update the order from PLACED → PREPARING → READY → COMPLETED

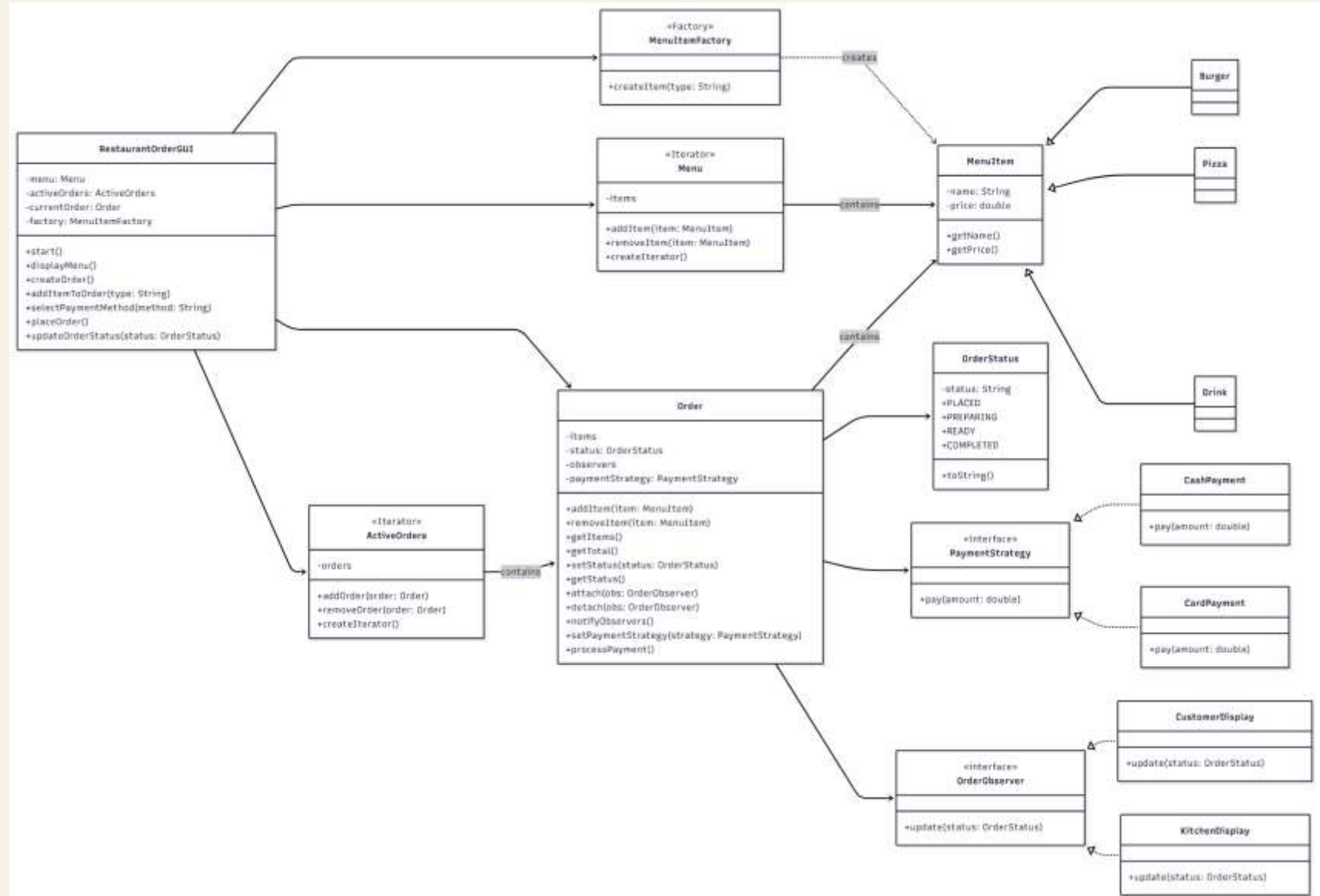
UML DIAGRAM

Main UML Structure

- RestaurantOrderGUI connects the user interface to the system logic
- Order stores selected items, payment method, status, and observers
- MenuItem is the parent class for Pizza, Burger, and Drink
- PaymentStrategy is the interface for CashPayment and CardPayment
- OrderObserver is the interface for status update displays

Patterns Used

- **Factory** → MenuItemFactory creates menu items
- **Strategy** → Order uses PaymentStrategy for payment
- **Observer** → Order notifies OrderObserver when status changes
- **Iterator** → Menu and ActiveOrders support item/order traversal



DESIGN PATTERNS: FACTORY + STRATEGY

FACTORY PATTERN

- **Purpose:** centralizes object creation for menu items
- **Class:** MenuItemFactory
- **Creates:** Pizza, Burger, or Drink based on selected type
- **Benefit:** GUI does not directly decide how each menu item object is built

Factory Pattern Example

```
MenuItem item = MenuItemFactory.createItem(type);
currentOrder.addItem(item);
updateOrderArea();
```

STRATEGY PATTERN

- **Purpose:** lets the order switch payment behavior
- **Interface:** PaymentStrategy
- **Implementations:** CashPayment and CardPayment
- **Benefit:** adding another payment method does not require rewriting Order

Strategy Pattern Example

```
public interface PaymentStrategy {
    boolean pay(double amount);
}
```

```
currentOrder.setPaymentStrategy(new CashPayment());
currentOrder.processPayment();
```

```
currentOrder.setPaymentStrategy(new CardPayment());
currentOrder.processPayment();
```

DESIGN PATTERNS: OBSERVER + ITERATOR

OBSERVER PATTERN

- **Purpose:** update displays when an order status changes
- **Subject:** Order stores a list of OrderObserver objects
- **Observer:** RestaurantOrderGUI implements OrderObserver
- Status changes call notifyObservers(), then update the GUI label

Observer Pattern Example

```
currentOrder = new Order();  
currentOrder.addObserver(this);
```

```
public interface OrderObserver {  
    void update(OrderStatus status);  
}
```

```
currentOrder.setStatus(OrderStatus.PREPARING);  
currentOrder.setStatus(OrderStatus.READY);  
currentOrder.setStatus(OrderStatus.COMPLETED);
```

ITERATOR PATTERN

- **Purpose:** browse menu items without showing internal list
- **Class:** MenuIterator implements Iterator<MenuItem>
- Menu traversal is separated from MenuItem and GUI code
- This keeps collection access clean and replaceable

Iterator Pattern Example

```
public class MenuIterator implements Iterator<MenuItem> {  
    private final List<MenuItem> items;  
    private int index = 0;  
    public MenuIterator(List<MenuItem> items) { this.items = items; }  
    @Override  
    public boolean hasNext() { return index < items.size(); }  
    @Override  
    public MenuItem next() { return items.get(index++); }  
}
```

GUI WINDOW DESIGN

The screenshot shows a window titled "Restaurant Order System" with standard Windows window controls (minimize, maximize, close). The interface is organized into several sections:

- Menu Items:** A list box containing "pizza", "burger", and "drink".
- Add Item:** A button located below the menu items list.
- Order Items:** A text input field for entering the order details.
- Order Status:** A label displaying "PLACED".
- Payment:** Two buttons, "Pay Cash" and "Pay Card", stacked vertically.
- Update Status:** A section containing three buttons: "Set PREPARING", "Set READY", and "Set COMPLETED".

On the far left edge of the window, there is a vertical list of labels: "Do", "Do", "Do", "Do", "Do", "Do".

PROGRAM DEMONSTRATION

Live demonstration

Launch app → select menu item → add to order → pay cash/card → update status → show label change

Demo Steps

1. Launch the JavaFX application
2. Select Pizza, Burger, or Drink from the menu list
3. Click Add Item to add it to the current order
4. Choose Cash or Card payment
5. Update the order status
6. Show the GUI label changing as the order progresses

GROUP MEMBER CONTRIBUTIONS

ADEN YU

- Created UML diagram draft
- Implemented Factory pattern
- Implemented Strategy pattern
- Helped with GUI components
- Set up GitHub repository

JUSTIN MABRY

- Implemented Observer pattern
- Implemented Iterator pattern
- Worked on order tracking/status updates
- Helped test and refine the GUI
- Worked on project report

IMPLEMENTATION TIMELINE

Phase 1: April 13–16:

- Finalized group and proposal
- Set up GitHub repository
- Created UML system draft
- Mapped design patterns and use cases

Phase 2: April 17–23:

- Set up Java project structure
- Implemented MenuItem classes and Factory pattern
- Began JavaFX GUI layout
- Created Order and Payment class structure

Phase 3: April 24–27:

- Integrated Strategy and Observer patterns
- Connected GUI to controller layer
- Tested order status updates
- Refined UML diagrams

Phase 4: April 28–29:

- Uploaded final code to GitHub
- Compiled final PDF
- Submitted project by deadline
- Present final project